内容
概述
前言
类型（GType）与对象（GObject）
对象的定义
信号的定义和应用
对象的属性和方法
对象的继承
对象的实现
对象的初始化
使用自定义类型
对象的析构
结语
相关资源

GObject对象系统

宋宝华
2007-8-8 11:00

前言

大多数的软件都在底层使用类似面向对象的编程模型来组织，例如图形、文件、设备以及硬件的抽象。这种面向对象的编程模型（对象、类、继承、信号、属性）可以帮助开发者创建高度可重用的、可扩展的、可维护的软件。GNU 项目使用 GTK+ 作为其图形工具包，它以及其它很多工具都使用 GNU 的 GLIB 工具包来实现这种面向对象的编程模型。GLIB 工具包提供了一个称为 GType 的类型系统，它实现了对这种模型的支持。

一个典型的面向对象的系统可能会包括下列这些概念：

- 一个层次结构的类，类可以继承，可以实现一个或多个接口。
- 一个基本的对象类，它为其子类提供一个通用的内存分配模型，对象的内存布局，对象的引用计数，对象的属性和信号的支持。
- 一个基本的接口类型，它为其子类提供一个通用的接口定义。

这种模型的优点在于程序员可以在这个通用的基础上创建一个高度可重用的、可扩展的、可维护的面向对象的软件。

- 一个可扩展的参数系统，它用特殊的数据结构来记录每个对象的实例所具有的属性。

类型（GType）与对象（GObject）

GLIB 中提供的类型系统的核心是 GObject 类型。它是以 GTypeInstance 为基类的一系列结构体的实现。GTypeInstance 的定义如下：

GTypeInstance 是所有对象实例的基类型。GType API 是 GObject 的基类型，所以每个 GObject 实例的第一个成员都是 GTypeInstance 。GTypeInstance 的成员仅仅用来记录一个指向对象类型结构体的指针。这种结构体一般称为对象的类结构体（class structure）。这种结构体的成员是一系列指向实现该类型的函数的指针。

为实现类型定义和注册功能，所有类型必须基类型的定义和注册函数。这些函数通常是静态的，并且通常以 class_init 和 base_init 作为前缀。例如，type_register_static() 函数以 GTypeInfo 结构体指针作为其参数传递给基类。这些函数包括 class_init、class_finalize、base_init、base_finalize 等函数。这些函数通常是静态的，并且通常以 class_init 作为前缀。这些函数用来初始化类结构体。

- 一个基本的类型注册机制，用来注册新的类型。

- 一个基本的类型实例分配机制，用来分配新的对象实例。

- 一个基本的类型实例初始化机制，用来初始化新的对象实例。

- 一个基本的类型实例析构机制，用来析构和释放对象实例。

- 一个基本的类型实例引用计数机制，用来管理对象的生命周期。

- 一个基本的属性机制，用来管理对象的属性。

- 一个基本的信号机制，用来管理对象的信号。

对象的定义

GObject 的基本，对象的定义通常包括：

- 对象的实例结构（class），实例变量，所有这些对象共用的数据；
- 对象的类结构（class，结构体），由对象的所有实例共用的数据；
- 对象的类型（type），对象的类型；

基于 GObject 的对象的实现通常会包括两部分：对象 API 是 GObject 的基类型，所以每个 GObject 实例的第一个成员都是 GTypeInstance 。GTypeInstance 的成员仅仅用来记录一个指向对象类型结构体的指针。这种结构体一般称为对象的类结构体（class structure）。这种结构体的成员是一系列指向实现该类型的函数的指针。

这是一个典型的对象实例结构体的定义：

```
/* Instance structure */
typedef _Foo Foo;
typedef struct _FooClass FooClass;
typedef struct _FooPrivate FooPrivate;
typedef enum _Foo_PROPERTIES_ENUM_PROTOTYPES
typedef enum _Foo_SIGNALS_ENUM_PROTOTYPES
typedef guint32 (*Foo_foo_func) (Foo *self, guint32 arg);
typedef void (*Foo_foo_signal_callbackvoid) (Foo *self, gpointer user_data);
struct _Foo {
  GTypeInstance parent_instance;
  gint ref_count;
  FooPrivate * priv;
  gint* _foo_array1;
  int _foo_array1_length1;
  int _foo_array1_length2;
  char* _foo_bar;
};

struct _FooClass {
  GTypeClass parent_class;
  void (*finalize) (Foo *self);
  guint32 (*foo) (Foo *self, guint32 arg);
  void (*foo_signal) (Foo *self, gint p1, gint p2);
  void (*foo_new_with_name_signal) (Foo *self, const char* p1);
  void (*foo_new_with_name2_signal) (Foo *self);
  void (*foo_new_with_name_int_signal) (Foo *self, const char* p1, gint p2);
  void (*foo_new_with_name_int_array_signal) (Foo *self, gint* p1, int p1_length1, gint p2);
  void (*_foo_2__foo_v) ();
};

gint _foo_get_foo1 (Foo * self);
void _foo_set_foo1 (Foo * self, gint value);
gint _foo_get_foo2 (Foo * self);
void _foo_set_foo2 (Foo * self, gint value);
gint _foo_get_foo3 (Foo * self);
void _foo_set_foo3 (Foo * self, gint value);
gint
```